

APPLIED DATA SCIENCE(LAB)

ASSIGNMENT NO : 3

TITLE : Assignment based on Implementation and Optimization of Ensemble Models, Case Study: Implement XGBoost for a Kaggle competition dataset (e.g., Titanic or House Price Prediction). Use xgboost, lightgbm, and scikit-learn for metrics and cross-validation

NAME: Shinde Mahesh Tulshiram

ROLL NO : 150

CLASS : TY-B

GITHUB LINK :

https://github.com/maheshshinde9100/SCOE_CSE_SEM_6/blob/master/AppliedDataScience/Assignment3.py

DATASET : California Housing Dataset

Code:

```
# =====
# Implementation and Optimization of Ensemble Models
# Case Study: XGBoost & LightGBM (Regression Problem)
# Dataset: California Housing Dataset
# =====

# =====
# 1. Import Libraries
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold, cross_val_score, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

import warnings
warnings.filterwarnings("ignore")

# =====
# 2. Load Dataset
# =====

file_path = "/content/california_housing.csv"
target_column = "medianHouseValue"
```

```

df = pd.read_csv(file_path)

print("\nDataset Shape:", df.shape)
print("\nFirst 5 Rows:\n", df.head())

# =====
# 3. Separate Features & Target
# =====

X = df.drop(columns=[target_column])
y = df[target_column]

# Handle missing values (if any)
X.fillna(X.median(), inplace=True)

# =====
# 4. Train-Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("\nTrain Shape:", X_train.shape)
print("Test Shape:", X_test.shape)

# =====
# 5. XGBoost Regressor
# =====

print("\n===== XGBOOST REGRESSOR =====")

xgb_model = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred_xgb)

```

```

mse = mean_squared_error(y_test, y_pred_xgb)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_xgb)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 6. Cross Validation
# =====

print("\n===== CROSS VALIDATION =====")

kf = KFold(n_splits=5, shuffle=True, random_state=42)

cv_scores = cross_val_score(
    xgb_model,
    X,
    y,
    cv=kf,
    scoring='r2'
)

print("Cross Validation R2 Scores:", cv_scores)
print("Mean CV R2:", cv_scores.mean())

# =====
# 7. Hyperparameter Tuning
# =====

print("\n===== GRID SEARCH OPTIMIZATION =====")

param_grid = {
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [200, 300]
}

grid_search = GridSearchCV(
    estimator=XGBRegressor(random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring='r2',
    n_jobs=-1
)

```

```

grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best CV R2 Score:", grid_search.best_score_)

# =====
# 8. LightGBM Regressor
# =====

print("\n===== LIGHTGBM REGRESSOR =====")

lgb_model = LGBMRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    random_state=42
)

lgb_model.fit(X_train, y_train)

y_pred_lgb = lgb_model.predict(X_test)

print("LightGBM R2:", r2_score(y_test, y_pred_lgb))
print("LightGBM RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lgb)))

# =====
# 9. Feature Importance
# =====

print("\n===== FEATURE IMPORTANCE =====")

importances = xgb_model.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10,6))
plt.title("Feature Importance - XGBoost")
plt.bar(range(X.shape[1]), importances[indices])
plt.xticks(range(X.shape[1]), X.columns[indices], rotation=90)
plt.tight_layout()
plt.show()

print("\nAssignment Completed Successfully ")

```

Output:

Dataset Shape: (20640, 9)

First 5 Rows:

	longitude	latitude	housingMedianAge	totalRooms	totalBedrooms \
0	-122.23	37.88	41	880	129
1	-122.22	37.86	21	7099	1106
2	-122.24	37.85	52	1467	190
3	-122.25	37.85	52	1274	235
4	-122.25	37.85	52	1627	280

	population	households	medianIncome	medianHouseValue
0	322	126	8.3252	452600.0
1	2401	1138	8.3014	358500.0
2	496	177	7.2574	352100.0
3	558	219	5.6431	341300.0
4	565	259	3.8462	342200.0

Train Shape: (16512, 8)

Test Shape: (4128, 8)

===== XGBOOST REGRESSOR =====

MAE: 32231.828166636387

MSE: 2335230480.4869742

RMSE: 48324.222502663964

R2 Score: 0.8217937667351913

===== CROSS VALIDATION =====

Cross Validation R2 Scores: [0.82077345 0.82980539 0.82393182 0.84742243 0.81665236]

Mean CV R2: 0.8277170885264755

===== GRID SEARCH OPTIMIZATION =====

Best Parameters: {'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 300}

Best CV R2 Score: 0.8298675761450648

===== LIGHTGBM REGRESSOR =====

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.001656 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 1838

[LightGBM] [Info] Number of data points in the train set: 16512, number of used features: 8

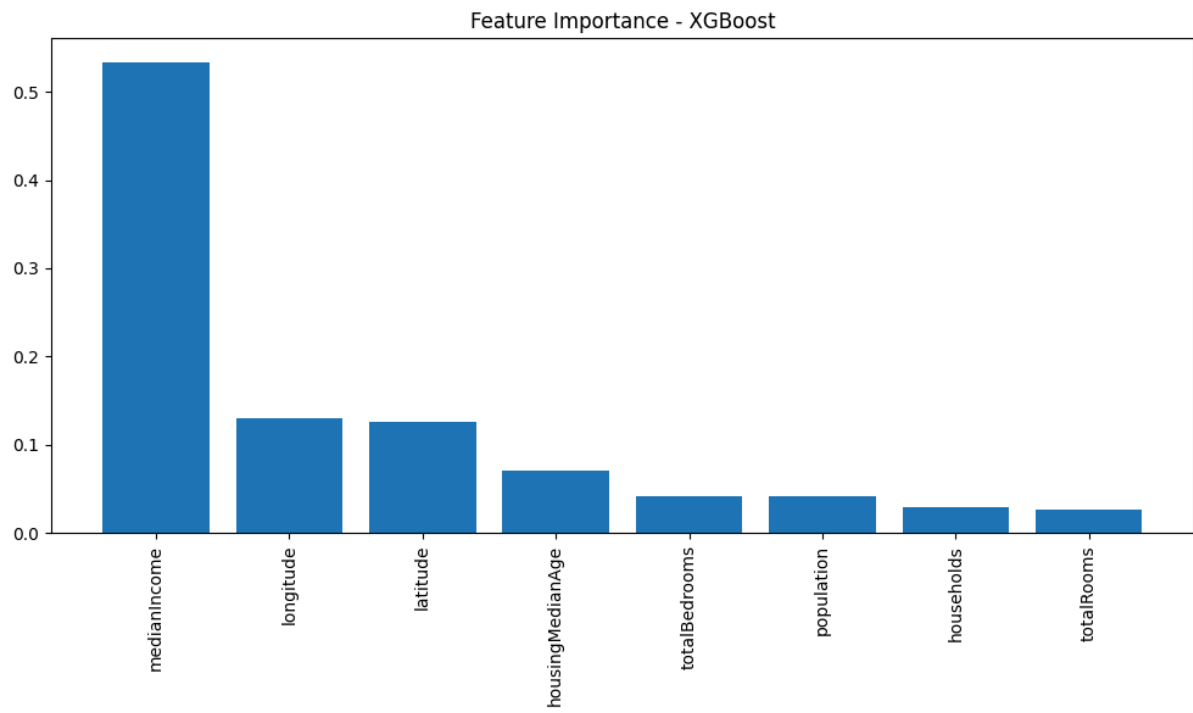
[LightGBM] [Info] Start training from score 207194.693738

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

LightGBM R2: 0.821518248889262

LightGBM RMSE: 48361.564191790654

===== FEATURE IMPORTANCE =====



Assignment Completed Successfully